



UNIVERSITATEA DIN BUCUREȘTI

FACULTATEA DE
MATEMATICĂ ȘI INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de disertație

EXPANDING ON DECLARATIVE PROCESS MINING WITH TAMARIN PROVER

Absolvent

Victor Andrei Cociobanu

Coordonator științific

Prof. dr. Ioana Gabriela Leustean

Bucharest, September 2026

Abstract

This dissertation presents an application of Tamarin prover to supplement conformance checking with flexible rules. Inspired by the work of `declare4py`, to use LTLf expressions to conformance check log traces, this paper seeks to observe whether the lack of a complete process model leaves room for undesirable behavior. The Tamarin prover is used to verify whether the process model that emerges from the log traces will still respect the given constraints. To do it, a short algorithm is proposed. It takes a series of traces within a log file and LTL expressions. The former are piped into `pm4py`, using the process mining feature, to build a process model graph. That will then be transformed into the process model for Tamarin. The latter are first ingested into `declare4py`, to perform a conformance check on the traces, then are in turn translated to Tamarin lemmas. Compiling both phases into a `spthy` file then allows Tamarin to prove or to find an example that falsifies the constraints, according to the mined process model.

Rezumat

Lucrarea prezinta o aplicatie a demonstratorului Tamarin pentru a suplimenta verificarea conformitatii prin reguli flexibile. Inspirat de abordarea `declare4py`, de a folosi expresii LTLf pentru a verifica conformitatea intrarilor din jurnalul unui proces, aceasta lucrare isi propune sa observe daca lipsa unui model de proces complet lasa loc pentru comportament indezirabil. Tamarin prover este folosit pentru a verifica daca modelul de proces care este construit din intrarile de jurnal va continua sa respecte constrangerile date. Pentru a face asta, un algoritm scurt este propus. Acesta ia o serie de intrari de jurnal si expresii LTL. Intrarile sunt oferite catre `pm4py`, folosind capabilitatea de a mina un proces, ca sa construiasca un graf al modelului de proces. Ulterior, graful, va fi transformat in modelul de proces pentru Tamarin. Expresiile sunt oferite catre `declare4py`, pentru a verifica conformitatea jurnalului. Ulterior expresiile sunt traduse in leme de Tamarin. Punand ambele parti intr-un fisier `spthy`, Tamarin este capabil sa demonstreze, sau sa gaseasca un contraexemplu care sa falsifice, constrangerile conform modelului de proces minat.

Contents

1	Introduction	6
1.1	Inquiry	6
1.2	Answer	6
1.3	Scope	7
1.4	Thesis structure	7
2	Automation	8
2.1	Flow	8
2.2	XES format	9
2.3	LTLf	9
2.4	pm4py	10
2.5	declare4py	10
2.6	Tamarin	11
2.6.1	Transition rule	11
2.6.2	Lemma	12
2.6.3	Restriction	12
3	Implementation	13
3.1	Guardedness	13
3.1.1	Nesting	13
3.2	Until	14
3.3	Transitions	14
3.4	Restrictions	15
3.4.1	Must end	15
3.4.2	Cycles	16
3.5	Lemmas	16
4	Example	19
4.1	Meaning	19

4.2	Conformance	20
4.3	Third	20
4.4	Tamarin	20
4.4.1	Lemma	20
4.4.2	Verification	21
4.5	Discussion	22
5	LLM Usage	23
5.1	Search	23
5.2	Infrastructure	23
5.3	Busywork	23
5.4	Thesis	23
6	Conclusion	24
	References	25
	Appendices	27
A	Example Event Log	27

1 Introduction

Most problems reduce to problems over canonic data structures. In the case of process models, logs, or verification and falsification for security protocols, the underlying data structure is a graph. Fundamentally though, they tackle two different problems, one takes a process model and traces, and verifies that the traces respect the model, while the other one takes the model, some properties and verifies that the model itself respects those properties (through the traces it can generate). For verifying traces against a model `pm4py` [2] is a robust tool. It offers a collection of features ranging from Process Modeling, Process Discovery, Process Simulation, to Conformance Checking, Visualization and others. Their Token-Based Replay algorithm is an intuitive approach that takes a concrete execution and simulates it over the process model to see if the allowed transitions happen. That approach is rigid, since it exhaustively defines the valid paths a process can take. A different approach is the focus of `declare4py` [5]. It builds its model from Linear Temporal Logic on finite traces formulas. Something similar to the security properties defined by a protocol prover. Defining only what is necessary to happen introduces flexibility. It allows for the construction of processes that adapt in undefined ways, while still ensuring key aspects are preserved.

1.1 Inquiry

Unlike a protocol prover, `declare4py` only has access to a finite number of traces. That invites the following question: Given only traces that pass the properties, will the process model resulting from them also pass the properties?

1.2 Answer

Intuitively no, joining two subgraphs can lead to more paths than the sum of the paths in the two subgraphs. And further in the paper a sample example will be explored.

1.3 Scope

As protocols benefit from provers to rule out certain forms of undesirable behavior, so can process models benefit from such verifications. Processes that don't have a definitive model, but instead are defined by a union of properties being the expected beneficiary. Having a lack of information about the future, by design, automating such activities is necessary to be able to track the evolution of such a process.

1.4 Thesis structure

This paper tackles the underlying work in distinct stages, that take the form of sections. In Automation 2 the strategy to transform from what process mining tools use, to what Tamarin prover uses is presented. Then in Implementation 3 relevant technical aspects are the focus. Example 4 provides a concrete, but hypothetical, situation to illustrate the outcome of this papers implementation. Finally, the LLM Usage 5 section discusses how such a tool was used in the making of this project.

2 Automation

In this chapter a high level overview of the transformation process is provided, as well as a presentation of the tools used, and the reason for their inclusion.

2.1 Flow

To make verifying with Tamarin a feasible process, it must not introduce a burdensome level of complexity. Therefore robust tools and standards will be used where applicable.

Two inputs are necessary for this activity:

- Logs, in XES format [11]
- Constraints, written as LTLf expressions [3].

As seen in the diagram 1, they are first used by process mining tools. pm4py [2] to build a Directly-Follows Graphs (DFG), which will serve as the process model. declare4py [5] to perform a conformance check against the traces themselves. Having traces that don't pass this check already shows undesirable behavior, making a further search redundant. It also serves to show that the two inputs are valid.

Focus for the next chapter are the two scripts that translate a DFG and LTLf expressions to Tamarin input. As a final step, Tamarin prover will be invoked.

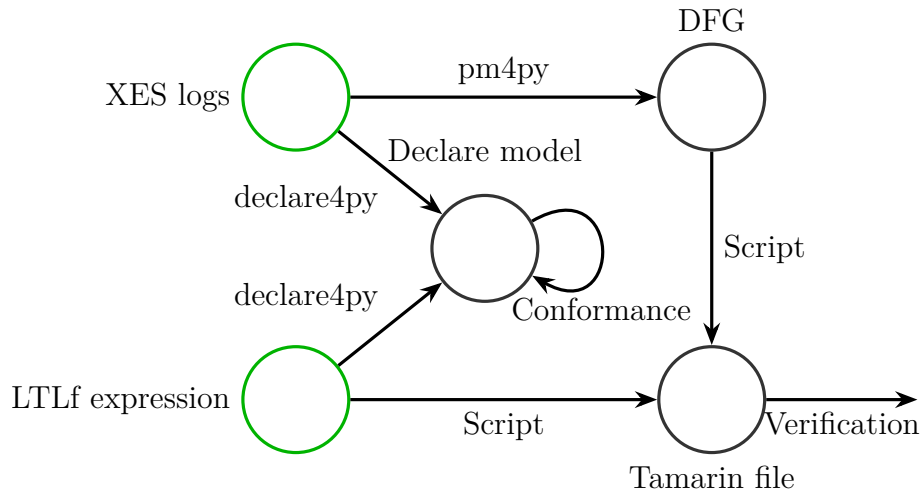


Figure 1: Workflow diagram.

2.2 XES format

It is an effort by IEEE [11] to standardize logging. Based on XML it aims to be Simple, Flexible, Extensible and Expressive. Broadly it represents a log object that contains an arbitrary number of traces, which in turn contain an arbitrary number of events. Since data by itself doesn't have meaning, the standard employs extensions. For the purpose of conformance checking, the Concept extension introduces the ability to label events. The log used for the example in this paper can be seen in the appendix A. It illustrates the nested nature of the data structure, and features key="concept:name", the element which names each event in a given trace, and each trace in a log.

2.3 LTLf

Linear Temporal Logic on finite traces [3]. A modal logic, that deals with discrete steps of time. It is capable of describing properties that must hold in regards to entire traces. For achieve that, the truth of a logic formula gets defined also in terms of the instance of time for which the evaluation happens. As an example the formulation of $\wedge$ by De Giacomo and Vardi [3] only introduces a named instance of time.

$$“\pi, i \models \varphi_1 \wedge \varphi_2 \text{ iff } \pi, i \models \varphi_1 \text{ and } \pi, i \models \varphi_2.”$$

Taking the first trace from the appendix A, with the first event to be at the instance of time $i = 0$, and the presence of a value for the “concept:name” field as the mechanism that satisfies its label we have:

- $\pi, 0 \models \textit{submit}$ as true.
- $\pi, 0 \models \textit{submit} \wedge \textit{manualinspection}$ as false
- $\pi, 1 \models \textit{manualinspection}$ as true.

This introduces the need for formulas that speak of different instances of time, in relation to i . The same paper [3] introduces the next operator:

$$“\pi, i \models \bigcirc \varphi \text{ iff } i < \textit{last} \text{ and } \pi, i + 1 \models \varphi.”$$

It speaks of the immediate instance of time, therefore $\pi, 0 \models submit \wedge \bigcirc manualinspection$ is true

2.4 pm4py

A robust tool for process mining [2] that operates on XES logs, and is capable of building Directly-Follows Graphs (DFG). It encapsulates related activities as well, most notable the ability to create a visualization of the process model.

```
import pm4py
def make_image(log_path: str, output_image_path: str):
    log = pm4py.read_xes(log_path)
    dfg, start_activities, end_activities = pm4py.discover_dfg(log)
    pm4py.save_vis_dfg(dfg, start_activities, end_activities,
                      output_image_path)
    print(f"DFG visual: {output_image_path}")
```

The figure 2 is an adaptation of the output image. It can also be found in the demo code [15], in the demodata folder. The above script also displays the trivial way in which this tool allows the process model to be discovered and converted to a graph.

2.5 declare4py

This tool employs LTLf expressions. They get transformed into a DFA [9], to loosely define a process model. It then can take XES logs, using “concept:name” as in the example from a previous section 2.3 to check them against the model [5] it builds. The project suggests filling in templates, their way of building higher level operations, with elements that will verify on the traces.

```
template = LTLTemplate('precedence')
activities_a = ["manualinspection"]
activities_b = ["manualissuance"]
model = template.fill_template(activities_a, activities_b)
```

If multiple events are passed to an operand of the template, they are joined with the disjunction symbol. The formula that this template produces will be presented further, in the example 4 section. Having a log file, and the models, running the check becomes

simple:

```
def run_check(log_path: str, ltl_models: list):
    event_log = D4PyEventLog()
    event_log.parse_xes_log(log_path)

    analyzer = LTAnalyzer(event_log, ltl_models)
    results_df = analyzer.run_multiple_models()

    print(results_df)
```

For each trace it prints whether all the models are satisfied by it.

2.6 Tamarin

A powerful protocol prover [12], that defines its model in terms of transition rules, and its properties in a guarded first-order logic. It provides a manual [13] that explains many more features than used for this paper. Relevant within scope are the more universal features: transition rules, restrictions, and lemmas. They contain states and events, which will also be key concepts.

2.6.1 Transition rule

Starting with the anatomy of a transition rule:

```
rule Start_submit :
  [ Fr(~fr) ]
  --[ Tamalways(~fr), Submit(~fr), Tamstart(~fr) ]->
  [ State_submit(~fr) ]
```

As an oversimplification of the description inside the manual [13]; A rule is made of three parts:

- The set of states that must exist for the transition to be able to happen.
- The set of events, things that restrictions and lemmas will quantify.
- The set of states that will exist after the transition executes. They can be states that are input for other transitions, or an empty set to end the execution.

More is to be said about the premise set of states. Since traces must start, the set can be empty. It does not provide much usefulness thus the manual [13] speaks of the “Fr fact”. Used in the example above, it marks the generation of random value. For the scope of this paper, it is sufficient to think of it as a start state which gives a unique trace id. It exists for the sole purpose of allowing the Submit event to be created.

2.6.2 Lemma

Called “trace properties” [13], they are first-order logic formulas.

```
lemma lemma_3:
  "All fr #t. Automaticissuance(fr)@t ==>
    (Ex #u. Automaticinspection(fr)@u & #u < #t)"
```

It resembles the definition of LTLf formulas from 2.3, as in instances of time are also present. The manual defines them as sets of traces. Keeping with the theme where fr is the identifier of a trace, the lemma asks for all possible traces if Automaticissuance appears on the trace Automaticinspection must also exist at a previous instance of time. Next chapter will show side by side a LTLf formula and its translation to Tamarin.

2.6.3 Restriction

A lemma that instead of being verified, is enforced as if it would be part of the model. It is used in this paper to enforce that traces must execute until the end state:

```
restriction Force_Trace_Completion:
  "not (Ex fr #tinit. Tamstart(fr)@#tinit &
    not (Ex #tend. Tamend(fr)@tend))"
```

3 Implementation

In this chapter the focus is set on the two paths that perform the translation, and a third, derived, aspect. In the `spty.py` code file, inside the demo folder [15], `demot.build_transitions` is the start point for the DFG to transition rule translation, while `ltrlf.build_lemma` is the start for LTLf formula to Tamarin lemma translation. First, the considerations taken when translating must be explained.

3.1 Guardedness

The manual [13] speaks on it: “To ensure guardedness, for universally quantified variables, one has to check that they all occur in an action constraint right after the quantifier and that the outermost logical operator inside the quantifier is an implication. For existentially quantified variables, one has to check that they all occur in an action constraint right after the quantifier and that the outermost logical operator inside the quantifier is a conjunction.” Since the events of the trace will be the actions in this context this sounds like a non issue. On the other hand, looking at the next operator again 2.3, it speaks of a instance of time being earlier than the last. Similar for the i . At the root of the formula it can be assumed that i refers to the start of the trace. A simple solution is to introduce a start event, and an end one, since Tamarin permits an arbitrary number of events on rules. For the scope of this paper they are chosen as `Tamstart` and `Tamend`.

3.1.1 Nesting

Since logic formulas are inductively defined, there is nothing preventing the argument for the next operator to be a disjunction. $\circ(\text{submit} \vee \text{manualinspection})$. Translating it as is will raise a guardedness error. To solve this issue, all instances of time that are not certainly a start or end of the trace will receive a generic name. This paper chooses `Tamalways` as the name. Since at this moment, there exists a name for every event in the trace, it is resonable to conclude that naming the start and end is redundant. They are redundant, but keeping them named keeps the resulting lemma more expressive and concise.

3.2 Until

This subsection serves the purpose of visualising the desired output of a translation, and will be the last subsection before discussing the implementation. For showcase, the until operator is chosen: $\varphi_1 \mathcal{U} \varphi_2$, that states that on a trace φ_1 must hold true until φ_2 turns true, reference more than the start or end of a trace. Taking not a not the formulation for $\varphi_1 \mathcal{U} \varphi_2$ by De Giacomo and Vardi [3]:

“ $\pi, i \models \varphi_1 \mathcal{U} \varphi_2$ iff for some j such that $i \leq j \leq \text{last}$, we have that $\pi, j \models \varphi_2$ and for all k , $i \leq k < j$, we have that $\pi, k \models \varphi_1$.”

Taking φ_1 as Automaticinspection and φ_2 as Automaticissuance, it can be observed that four instances of time will need to be guarded. i will be considered the start, last is the end, and j and k will be guarded by the generic event. Given that the relationship between instances of time is given by the definition, the rest of the translation becomes natural:

```
lemma lemma_until:
  "All fr #i. Tamstart(fr) @i ==> (
    Ex #j #last . Tamalways(fr)@j & Tamend(fr)@last &
    ( #i < #j & #j < #last | #i = #j | #j = #last ) &
    Automaticissuance(fr)@j & (
      All #k . Tamalways(fr)@k &
      (#i < #k & #k < #j | #i = #k) ==> Automaticinspection(fr)@k
    ))"
```

3.3 Transitions

The first of the 2 mentioned flows starts with pm4py building the DFG. Not necessarily related, but here are also set the name choices mentioned in considerations.

```
pm4py_log = pm4py.read_xes(log_path)
dfg, start_acts, end_acts = pm4py.discover_dfg(pm4py_log)
fr_name = 'fr'
always_action = 'TamAlways'
start_action = 'TamStart'
end_action = 'TamEnd'

transitions = demot.build_transitions(dfg, start_acts, end_acts,
  fr_name, always_action, start_action, end_action)
```

Recalling the look of a transition rule 2.6.1, the following code snippet should look familiar:

```
for act in start_activities:
    rules.append(
        f"rule Start_{act}:\n"
        f"    [ Fr(~{fr_name}) ]\n"
        f"    --[ {always_action.capitalize()}(~{fr_name}), {act."
        f"        capitalize()}(~{fr_name}), {start_action.capitalize()}("
        f"            {~{fr_name}}) ]->\n"
        f"    [ State_{act}(~{fr_name}) ]\n"
    )
```

It is responsible for choosing names for the rule, actions and states dependent on the current action in the DFG. Since rules should not have conflicting names, a suffix dependent on the action is added. Action names get capitalized because a lowercase name issues a compilation error. Finally, states also get a personal suffix to ensure proper routing in the model is preserved. Similar steps are taken for the other type of nodes in the DFG, in the transitions.py script of the demo [15].

3.4 Restrictions

Not necessarily a part of the two flows, it is considered as an auxiliary script, the loops.py file. It exists because Tamarin can end trace generation early, and get stuck searching within a cycle.

3.4.1 Must end

A straight-forward requirement. It asks Tamarin to only consider the empty trace, or traces that reach the end.

```
def build_must_end_restriction(fr_name: str, start_action: str,
    end_action: str) -> str:
    return (
        f"// 5. Complete traces\n"
        f"restriction Force_Trace_Completion:\n"
        f'    not (Ex {fr_name} #tinit. {start_action.capitalize()}({fr_name})@#tinit &\n'
        f'        not (Ex #tend. {end_action.capitalize()}({fr_name})@tend))"\n'
    )
```

3.4.2 Cycles

Finding cycles inside a graph is a problem that NetworkX [10] solves. To break said cycle it is sufficient to remove one of the nodes. Looking at a restriction for only 2 executions:

```
restriction Max_2_Inspect:
    "not (Ex fr #t1 #t2 #t3.
        Inspect(fr)@#t1 &
        Inspect(fr)@#t2 &
        Inspect(fr)@#t3 &
        #t1 < #t2 &
        #t2 < #t3)"
```

Similar to the unique restriction from the manual [13], just written under the form of not exists. The code for `build_loop_restrictions` function implements this restriction for one node in each cycle. In a less reader friendly way, it prepares `n` instances of time, as many as requested by the caller, to be added after the `Ex` keyword. It also prepares the comparisons for the sequence of instances of time, since time is a well-ordered set with respect to the less than operation. It eventually has to loop over the traces, and name the event for which this restriction is created, with each instance of time, and put them together.

3.5 Lemmas

Traversing an expression tree has already been researched [1]. Given that `declare4py` maintains a `pylogics` [7] formula, the parsing step gets skipped. It must be noted that `Spot`[6], is a project capable of simplifying LTL formulas, employed here to try and reduce the complexity of the resulting lemma. Starting with traversing the expression tree in the `ltlf.py` file of the `demo` folder [15] `build_lemma` is the entry point. It is responsible for building the top level of the lemma.

```

rules = [
    f'// original: {model.formula}\n'
    f'// simpnorm: {spot_formula.to_str()}\n'
    f'lemma {lemma_name}:\n'
    f'    "All {fr_name} #{root_timestep}. {start_action.capitalize()}({fr_name}) @ {root_timestep} ==> (\n'
    f'        {translate_formula(formula, fr_name, root_timestep,
    f'            root_timestep + 't', always_action, end_action)}\n'
    f'    )" \n']

```

It makes a couple of choices. To consider all traces, and to set the top reference instance of time at the start of the trace. It then calls the recursive function of `translate_formula`. The higher level in the recursion is also responsible for choosing the root name of the instance of time variable for lower levels, as to avoid repetition when branching. Notable sections of the code are how it directly writes, following the definition [3], the text, for operators like `until`:

```

if isinstance(formula, Until):
    sub_l_ltl = factory_ltl(left_op, current_timestep + 'k',
        current_timestep + 'kl')
    sub_r_ltl = factory_ltl(right_op, current_timestep + 'j',
        current_timestep + 'jr')
    return (
        f'(Ex #{current_timestep}j #{current_timestep}last . {
            always_action.capitalize()}({fr_name})@{current_timestep}j &
            {end_action.capitalize()}({fr_name})@{current_timestep}last
            &' # set up j and last
        f' ( #{parent_timestep} < #{current_timestep}j & #{
            current_timestep}j < #{current_timestep}last | #{
            parent_timestep} = #{current_timestep}j | #{current_timestep}
            }j = #{current_timestep}last)' # state j between i and last
        f' & {sub_r_ltl} & '
        f' (All #{current_timestep}k . {always_action.capitalize()}({fr_name})@{current_timestep}k & (#{parent_timestep} < #{
            current_timestep}k & #{current_timestep}k < #{
            current_timestep}j | #{parent_timestep} = #{current_timestep}
            }k)' # i <= k < j
        f' ==> {sub_l_ltl} )'
    )

```

That it ends with an atomic:

```

if isinstance(formula, Atomic):
    return f'{formula.name[4:].capitalize() if formula.name[4] == '
        con_' else formula.name.capitalize()}({fr_name})@{
        parent_timestep}'

```

And that it uses logical equivalences:

```
if isinstance(formula, WeakUntil):
    sub_l_basic = factory_basic(Until(left_op, right_op))
    sub_r_basic = factory_basic(Always(left_op))
    return f"({sub_l_basic} | {sub_r_basic})"
```

4 Example

In this chapter an example will be presented, with the hypothetical flow from figure 2 below.

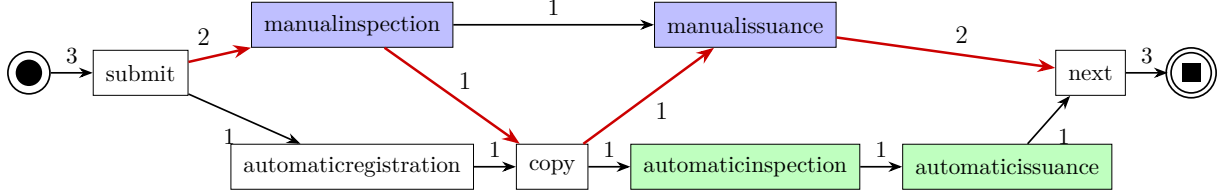


Figure 2: Two different flows with a trace jumping between them

Figure 2 is built based on the image that pm4py outputs for the log in the appendix A. The colored events take part in two LTLf formulas built with the precedence template shown in the previous section 2.5 With it two formulas are defined:

- $((\neg \text{manualissuance}) \mathcal{U} \text{manualinspection}) \vee \Box(\neg \text{manualissuance})$
- $((\neg \text{automaticissuance}) \mathcal{U} \text{automaticinspection}) \vee \Box(\neg \text{automaticissuance})$

Where $\Box$ is the operator for always [3], which states that the argument must hold true irrespective of the instance of time i . Thus the formulas would translate as issuance must not happen until inspection happens, or issuance must not ever happen.

4.1 Meaning

This flow would like to express that users can submit documents that will either follow a manual or an automatic action flow. For the manual path a user is expected to submit, then a manual inspection happens, which leads to a manual issuance of the result, for the next steps. The automatic path has more steps in that after submission, a copy is made, before being automatically inspected, and being issued the result, and rejoining the other flow for the next steps. Both paths express the desired behavior.

4.2 Conformance

Running the check with `declare4py` [5] it outputs:

	<code>case:concept:name</code>	<code>accepted</code>
0	Case 1	True
1	Case 2	True
2	Case 3	True

As expected, all traces pass the check, all of them have an inspection before issuance.

4.3 Third

The third trace in the appendix log; `<string key="concept:name" value="Case 3"/>` A presents an event chain that between a manual inspection and its issuance a copy is made. This action appears innocent, especially since the desired behavior is preserved.

4.4 Tamarin

In the online code [15] inside the `demodata` folder `process_theory.spthy` is the generated Tamarin file. It lacks the loop restrictions, since this model does not have any loops.

4.4.1 Lemma

Before analyzing one of the lemmas, it should be noted that `W` is the operation for weak until. It behaves like until, except the second operand does not need to happen. Its logical equivalence is shown in the implementation section 3.5. And the `con_` prefix is just an artefact of using the template.

```
// original: ((!con_manualissuance))U(con_manualinspection) || G(!con_manualissuance)
// simpnorm: !con_manualissuance W con_manualinspection
lemma lemma_1:
  "All fr #t. Tamstart(fr) @t ==> (
    ((Ex #ttj #ttlast . Tamalways(fr)@ttj & Tamend(fr)@ttlast & ( #
      t < #ttj & #ttj < #ttlast | #t = #ttj | #ttj = #ttlast) &
      Manualinspection(fr)@ttj & (All #ttk . Tamalways(fr)@ttk &
      (#t < #ttk & #ttk < #ttj | #t = #ttk) ==> not(
      Manualissuance(fr)@ttk)) )
    | (All #tt. Tamalways(fr)@tt & (#t < #tt | #t = #tt) ==> not(
      Manualissuance(fr)@tt)))
  )"

```

Fate has it happen, that even if the reduced formula is with weak until, because of using the equivalence for implementation, the lemma follows the original form. Three elements of its structure can be observed:

- The outer layer, which sets up that Tamarin to search for all traces from the start.
- The translation for until on the left side of the disjunction.
- The translation of Always on the right side.

4.4.2 Verification

Tamarin, does search for hypothetical traces that are not written into the log. At the end of its output it prints:

```
summary of summaries:
analyzed: demodata/process_theory.spthy

processing time: 0.62s

lemma_1 (all-traces): falsified - found trace (8 steps)
lemma_2 (all-traces): verified (10 steps)

```

It finds a trace that invalidates if manualissuance exists, it must be preceded by manualinspection. To look at the trace, or analyze further, the manual [13] explains how to use the interactive mode. The trace it finds passes from automaticregistration to copy and manualissuance, the red lines from the figure 2.

4.5 Discussion

The trace that invalidates a lemma, also invalidates the conformance check. It stands to reason that this information will eventually be found with enough executions. The utility of using Tamarin prover stands in its ability to preempt undesirable behavior. Finding it early allows for time to solve the issue before it can produce damages.

5 LLM Usage

As a lossy compression of all human knowledge, such a tool proves useful [14]. It was used with sufficient success for the project underlying this paper as well. The usage and the degree of usefulness can be put into a few categories.

5.1 Search

This may be the most successful application of such tools. Avoiding reimplementing studied algorithms, and dealing with edge cases that others spent time solving. This is how Spot [6] and NetworkX [10] were found by the description of a specific need.

5.2 Infrastructure

Most useful timesave yet, is using an LLM to setup infrastructure. Dependencies like Lydia [8] for declare4py [5], or even Spot [6] which is not distributed through the standard mechanism. Orchestrating all of them inside the work environment, in this case Dev Container [4], enabled the use of an unfamiliar tool that greatly reduced friction.

5.3 Busywork

Early iterations of the code were also mostly written by an LLM, doing the skeletal work of quickly patching up a sphinx file that compiles. Even more useful, generating log files for various examples.

5.4 Thesis

Fortunately, it struggled to implement the actual translation to Tamarin properties. Although presenting an interesting approach, in using a single event that differs in argument instead of name, it proved difficult in debugging. Especially when working with cycles inside the graph. Using different event names to force compiler errors, instead of runs not ending, proved as the better approach.

6 Conclusion

Identifying various tools, and the problems that they solve, with canonic data structures allows for their repurposing in different domains, with complementary value, to solve interesting problems. This paper shows that graph problems, in this case, allow for applying security protocol proving tools to be used in the domain of process mining. It must be said that a security protocol could also be considered a process, after all, SAPIC [13] allows protocols to be written with a process modeling language. Using that language would prove more cumbersome given the ease with which a DFG can be transformed into Tamarin rules. What made this paper possible is that the guarded first-order logic that Tamarin uses is much more flexible than LTLf, since going the other way would prove much more difficult. The most difficult challenge of this project, proposing how to guard the LTLf formulas, leaves experiments on the table. The exists-trace mode for lemmas [13] is not used here. It can build logically equivalent formulas, but it may be useful for projects that want to find if traces with certain properties can exist.

References

- [1] A. V. Aho and S. C. Johnson. “Optimal Code Generation for Expression Trees.” In: J. ACM 23.3 (July 1976), pp. 488–501. ISSN: 0004-5411. DOI: [10.1145/321958.321970](https://doi.org/10.1145/321958.321970). URL: <https://doi.org/10.1145/321958.321970>.
- [2] Alessandro Berti, Sebastiaan van Zelst, and Daniel Schuster. “PM4Py: A process mining library for Python.” In: Software Impacts 17 (2023), p. 100556. ISSN: 2665-9638. DOI: <https://doi.org/10.1016/j.simpa.2023.100556>. URL: <https://www.sciencedirect.com/science/article/pii/S2665963823000933>.
- [3] Giuseppe De Giacomo and Moshe Vardi. “Linear temporal logic and Linear Dynamic Logic on finite traces.” In: IJCAI International Joint Conference on Artificial Intelligence (Jan. 2013), pp. 854–860.
- [4] Development Containers Community. Development Container Specification. 2022. URL: <https://containers.dev/> (visited on 08/30/2026).
- [5] Ivan Donadello, Francesco Riva, Fabrizio Maria Maggi, and Aladdin Shikhizada. “Declare4Py: A Python Library for Declarative Process Mining.” In: BPM (PhD/Demos). Vol. 3216. CEUR Workshop Proceedings. CEUR-WS.org, 2022, pp. 117–121.
- [6] Alexandre Duret-Lutz. “Manipulating LTL Formulas Using Spot 1.0.” In: Automated Technology for Verification and Analysis. Ed. by Dang Van Hung and Mizuhito Ogawa. Cham: Springer International Publishing, 2013, pp. 442–445. ISBN: 978-3-319-02444-8.
- [7] Marco Favorito and contributors. pylogics: A Python library for logic formalisms representation and manipulation. <https://github.com/whitemech/pylogics>. 2020.
- [8] Marco Favorito and Giuseppe De Giacomo. “Lydia: A Tool for Compositional LTL f /LDL f Synthesis.” In: URL: <https://api.semanticscholar.org/CorpusID:268858832>.
- [9] Francesco Fuggitti. LTLf2DFA. Version 1.0.3. Mar. 2019. DOI: [10.5281/zenodo.3888410](https://doi.org/10.5281/zenodo.3888410).

- [10] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. “Exploring Network Structure, Dynamics, and Function using NetworkX.” In: Python in Science Conference (2008). DOI: [10.25080/TCWV9851](https://doi.org/10.25080/TCWV9851). URL: <https://doi.org/10.25080/TCWV9851>.
- [11] “IEEE Standard for eXtensible Event Stream (XES) for Achieving Interoperability in Event Logs and Event Streams.” In: IEEE Std 1849-2023 (Revision of IEEE Std 1849-2016) (2023), pp. 1–55. DOI: [10.1109/IEEESTD.2023.10267858](https://doi.org/10.1109/IEEESTD.2023.10267858).
- [12] Simon Meier, Benedikt Schmidt, Cas Cremers, and David Basin. “The TAMARIN Prover for the Symbolic Analysis of Security Protocols.” In: Computer Aided Verification. Ed. by Natasha Sharygina and Helmut Veith. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 696–701. ISBN: 978-3-642-39799-8.
- [13] The Tamarin Team. The Tamarin Prover User Manual. Tech. rep. ETH Zurich and CISP Helmholtz Center for Information Security, 2024. URL: <https://tamarin-prover.com/manual> (visited on 08/30/2026).
- [14] Linus Torvalds. Re: Linking Patchwork with Sashiko? July 14, 2026. URL: https://lore.kernel.org/linux-media/CAHk-=wi4zC+Ze8e+p3tMv8TtG_80KzsZ1syL9anBtmEh5Z40vg@mail.gmail.com/ (visited on 08/30/2026).
- [15] Victor. LTLfTam. <https://github.com/Vikcoc/LTLfTam>. 2026.

Appendices

This part holds code necessary to explain the paper, but that is too long to fit neatly inside the text.

A Example Event Log

The following code is an xes log used for this paper's example. It illustrates the construction of a log, with the `<log>` tag, `<trace>`, and `<event>`. Three traces can be observed, those are the ones used to build the process model, and which are verified by `declare4py`.

```
<?xml version="1.0" encoding="UTF-8" ?>
<log xes.version="1.0" xes.features="nested-attributes" xmlns="http://www.xes-standard.org">
  <extension name="Concept" prefix="concept" uri="http://www.xes-standard.org/concept.xesext"/>
  <extension name="Lifecycle" prefix="lifecycle" uri="http://www.xes-standard.org/lifecycle.xesext"/>
  <extension name="Time" prefix="time" uri="http://www.xes-standard.org/time.xesext"/>
  <classifier name="Activity classifier" keys="concept:name"/>
  <string key="concept:name" value="Process Log"/>

  <trace>
    <string key="concept:name" value="Case 1"/>
    <event>
      <string key="concept:name" value="submit"/>
      <string key="lifecycle:transition" value="complete"/>
      <date key="time:timestamp" value="2026-08-23 T09:00:00.000+00:00" />
    </event>
    <event>
      <string key="concept:name" value="manualinspection"/>
      <string key="lifecycle:transition" value="complete"/>
      <date key="time:timestamp" value="2026-08-23 T09:15:00.000+00:00" />
    </event>
    <event>
      <string key="concept:name" value="manualissuance"/>
      <string key="lifecycle:transition" value="complete"/>
      <date key="time:timestamp" value="2026-08-23 T09:30:00.000+00:00" />
    </event>
    <event>
      <string key="concept:name" value="next"/>
      <string key="lifecycle:transition" value="complete"/>
      <date key="time:timestamp" value="2026-08-23 T09:45:00.000+00:00" />
    </event>
```

```

    </event>
</trace>

<trace>
  <string key="concept:name" value="Case 2"/>
  <event>
    <string key="concept:name" value="submit"/>
    <string key="lifecycle:transition" value="complete"/>
    <date key="time:timestamp" value="2026-08-23 T10:00:00.000+00:00"
      >/>
  </event>
  <event>
    <string key="concept:name" value="automaticregistration"/>
    <string key="lifecycle:transition" value="complete"/>
    <date key="time:timestamp" value="2026-08-23 T10:10:00.000+00:00"
      >/>
  </event>
  <event>
    <string key="concept:name" value="automaticcopy"/>
    <string key="lifecycle:transition" value="complete"/>
    <date key="time:timestamp" value="2026-08-23 T10:20:00.000+00:00"
      >/>
  </event>
  <event>
    <string key="concept:name" value="automaticinspection"/>
    <string key="lifecycle:transition" value="complete"/>
    <date key="time:timestamp" value="2026-08-23 T10:30:00.000+00:00"
      >/>
  </event>
  <event>
    <string key="concept:name" value="automaticissuance"/>
    <string key="lifecycle:transition" value="complete"/>
    <date key="time:timestamp" value="2026-08-23 T10:40:00.000+00:00"
      >/>
  </event>
  <event>
    <string key="concept:name" value="next"/>
    <string key="lifecycle:transition" value="complete"/>
    <date key="time:timestamp" value="2026-08-23 T10:50:00.000+00:00"
      >/>
  </event>
</trace>

<trace>
  <string key="concept:name" value="Case 3"/>
  <event>
    <string key="concept:name" value="submit"/>
    <string key="lifecycle:transition" value="complete"/>
    <date key="time:timestamp" value="2026-08-23 T11:00:00.000+00:00"
      >/>
  </event>
  <event>
    <string key="concept:name" value="manualinspection"/>
    <string key="lifecycle:transition" value="complete"/>

```

```

    <date key="time:timestamp" value="2026-08-23T11:15:00.000+00:00
    "/>
  </event>
  <event>
    <string key="concept:name" value="automaticcopy"/>
    <string key="lifecycle:transition" value="complete"/>
    <date key="time:timestamp" value="2026-08-23T11:30:00.000+00:00
    "/>
  </event>
  <event>
    <string key="concept:name" value="manualissuance"/>
    <string key="lifecycle:transition" value="complete"/>
    <date key="time:timestamp" value="2026-08-23T11:45:00.000+00:00
    "/>
  </event>
  <event>
    <string key="concept:name" value="next"/>
    <string key="lifecycle:transition" value="complete"/>
    <date key="time:timestamp" value="2026-08-23T12:00:00.000+00:00
    "/>
  </event>
</trace>
</log>

```